MDM Autochecks

Headless pipeline that consolidates Business Partner identifiers, checks SIREN/SIRET via the INSEE SIRENE API, validates VAT numbers with the EU VIES batch service, and mails Excel reports for anomalies.

Features

- One run triggers both flows in parallel: SIREN/SIRET verification and VAT batch validation.
- Builds a clean partner dataset from SAP exports (pivot FR0/FR1/FR2 codes, normalize IDs, merge names and address tables) into `latest_datas.xlsx`.
- SIRENE lookups flatten legal unit and establishment data, add duplicate and mismatch flags, and write a color-coded `latest_report.xlsx`.
- VAT flow reformats values into batches of 100, uploads to VIES, polls tokens, downloads per-batch Excel files, concatenates them, and reattaches BP numbers.
- Email step (Graph API + certificate auth) sends focused extracts: inactive SIRET/SIREN, duplicate SIRET, fuzzy name issues, invalid VATs.

How it works (pipeline)

1. **Load inputs**: `.env` supplies paths. `main.py` reads the raw tax file (`INPUT_FILE` in `INPUTS`), keeps columns `[BP, value, type]` , pivots FR0 -> `VAT` , FR1 -> `siret` , FR2 -> `siren` , and maps EU VAT codes ending in `0` to FR0.
2. **Enrich partners** (`forSirenSiret.partner_processing.build_partner_dataset`): normalize IDs, merge the names file (`NAMES_FILE`), join address metadata (`JOIN_TABLE` , `ADRESS_TABLE`), detect duplicates/invalid lengths, and write `latest_datas.xlsx` under the dated output folder.
3. **SIREN/SIRET checks** (`forSirenSiret.checks.generate_report`): decide SIREN vs SIRET vs both based on consistency, retry network errors, and append rows with conditional formatting into `siren_siret/latest_report.xlsx` (diagnostics include fuzzy name match).
4. **VAT checks** (`forVats.process.process`): split the `VAT` column into CSV batches of 100, submit to VIES, poll tokens until `COMPLETED` , download each report, concatenate into `vat/report_concatenated.xlsx` , and add back BP IDs.
5. **Mail exports** (`emailing.mail_export`): build targeted extracts (`closed_siret.xlsx` , `closed_siren.xlsx` , `duplicated_siret.xlsx` , `wrong_name.xlsx` , `bad_vats.xlsx`) and send them from the shared mailbox.

Expected inputs

- `.env` at repo root:
  - `DIRECTORY_LOCATION` : base folder where dated run directories are created.

- `INPUTS` : directory containing the raw CSVs.
- `INPUT_FILE` : tax-number export (e.g., `BP_TAXNUM.csv` , columns: BP id, value, spec code such as FR0/FR1/FR2 or EU VAT codes ending with `0`).
- `NAMES_FILE` : BP info export (e.g., `BP_BUT000.csv` , must include `Business Partner` , `Name` , `Country/Region Key` , etc.).
- `JOIN_TABLE` : mapping BP -> address id (e.g., `BP_BUT020.csv`).
- `ADRESS_TABLE` : address table (e.g., `BP_ADRC.csv`).

- Optional: `emailing/config.cfg` or `config.dev.cfg` for Azure Graph (tenant/client IDs, PFX path/password).

Input shape (SAP CSV)

Typical formats read by `main.py` / `handcheck.py` :

- Without headers (`;` separator):

```
BP      ; value        ; type ; UNKNOWN TEXT
900080  ; FR25784158545; FR0  ; 0
900080  ; 7,84E+08     ; FR2  ; 0
900080  ; 7,84E+13     ; FR1  ; 0
```

- With French headers (auto-detected delimiter, columns like “Partenaire”, “Cat. N° ID fiscale”, “N° ID fiscale”, “N° ID fiscale (long)”, …):

```
Partenaire,Cat. N° ID fiscale,N° ID fiscale,N° ID fiscale (long),...
1000915,FR0,FR95790959096,,
1000915,FR1,7909950900019,,
1000915,FR2,790995096,,
```

Only the first 3 columns are used and renamed `BP` , `type` , `value` . EU codes ending with `0` are mapped to `FR0` to feed the `VAT` column.

Outputs

- `<DIRECTORY_LOCATION>/<timestamp>/latest_datas.xlsx` : consolidated partner lines with normalized IDs, duplicates, address flags, and BP metadata.
- `.../siren_siret/latest_report.xlsx` : SIREN/SIRET API results with status, NAF, address, duplicate lists, and diagnostic columns (`report` , `diagnostic_name`).
- `.../vat/report_concatenated.xlsx` : merged VIES results with original source file names and BP linkage.
- `.../siren_siret/closed_siret.xlsx` , `.../closed_siren.xlsx` , `.../duplicated_siret.xlsx` , `.../wrong_name.xlsx` , `.../vat/bad_vats.xlsx` : anomaly extracts used by the mailer.

Output file structure (key columns)

- `latest_datas.xlsx` (Sheet1):
  - Identifiers: `BP`, `siren`, `siret`, `VAT`
  - Flags: `duplicates_siren`, `duplicates_siret`, `duplicates_VAT`, `missing siren`, `missing siret`, `Missing vat`, `Missmatching siren siret`, `Missmatching siren VAT`, `uses a snetor *`
  - BP info / address: `Name 1`, `Grouping`, `Country/Region Key`, `Language Key`, `adressID`, `street`, `street4`, `street5`, `city`, `postcode`, `country`, `has_*`
- `siren_siret/latest_report.xlsx` (sheet `Report`):
  - Identifiers: `BP`, `type` (siren/siret), `siret`, `nic`, `siren`
  - INSEE data: `status`, `siege`, `denomination`, `date_creation`, `date_cessation`, `naf`, `naf_label`, `cat_juridique`, `adresse`, `n_voie`, `voie`, `code_postal`, `commune`, `siret_siege`
  - Local checks: `duplicates_*`, `missing *`, `uses a snetor *`, `Missmatching siren *`
  - Summary: `report`, `diagnostic_name`
- `vat/report_concatenated.xlsx`:
  - VIES columns: `MS Code`, `VAT Number`, `Requester MS Code`, `Requester VAT Number`, VIES validity/status fields
  - Local additions: `__source_file__` (origin batch), `BP` (list of BPs matching the VAT)

Output tree

```
YYYY-MM-DD_HH-MM_REPORT/
├─ latest_datas.xlsx           # consolidated partners
├─ siren_siret/
│  └─ latest_report.xlsx       # INSEE results with conditional formatting
├─ vat/
│  ├─ data/                    # CSV batches sent to VIES
│  ├─ reports/                 # downloaded VIES Excel reports
│  ├─ report_concatenated.xlsx # merged VIES reports
│  └─ tokens.csv               # batch_file -> token mapping
└─ (handcheck adds suffix `_HANDCHECK_REPORT`)
```

Workflow (summary)

1. Select the SAP CSV (or run `main.py` to pick `INPUT_FILE` from `.env`).
2. The script pivots FR0/FR1/FR2 (and EU `*0` codes) to `VAT` / `siret` / `siren`, normalizes, and enriches.
3. Parallel launch of SIRENE (SIREN/SIRET) and VIES (VAT) checks.
4. Excel outputs are written to the dated folder (`..._REPORT` or `..._HANDCHECK_REPORT`), then mail exports run.

Expected result (example)

```
2025-12-22_11-57_REPORT/
├─ latest_datas.xlsx
├─ siren_siret/
│  └─ latest_report.xlsx
├─ vat/
│  ├─ data/
│  │  ├─ BP_TAXNUM_part000.csv
│  │  ├─ BP_TAXNUM_part001.csv
│  │  └─ ...
│  ├─ reports/
│  │  ├─ BP_TAXNUM_part000_report.xlsx
│  │  ├─ BP_TAXNUM_part001_report.xlsx
│  │  └─ ...
│  ├─ report_concatenated.xlsx
│  └─ tokens.csv
└─ (closed/duplicate/etc. extracts are emailed when present)
```

Handcheck mode (manual sample)

- GUI script: `handcheck.py` (Tkinter). Lets you pick any SAP-format BP CSV and run the full pipeline on a small sample without touching the scheduled batch inputs.
- What it does:
  - Prompts for the CSV (defaults to `INPUTS` path).
  - Auto-detects delimiter/header shapes, pivots FR0/FR1/FR2 (and EU `*0` codes -> FR0) into `VAT` / `siret` / `siren`.
  - Reuses the same enrichment/duplicate logic, then launches SIRENE and VIES flows in parallel threads.
  - Writes results to a dated folder under `DIRECTORY_LOCATION` with suffix `_HANDCHECK_REPORT`, then triggers mail exports.
- How to run:

  ```
  python handcheck.py
  ```

  Or open the executable to run it without installing dependencies.
 Click **Pick SAP BP CSV and run**, select the sample file, and watch the live log. Outputs land in `Z:\MDM\998_CHecks\<timestamp>_HANDCHECK_REPORT\...`.

API endpoints used

- INSEE SIRENE: `GET https://api-avis-situation-sirene.insee.fr/identification/siren/{siren}?telephone=` and `GET https://api-avis-situation-sirene.insee.fr/identification/siret/{siret}?telephone=`.
- EU VIES batch VAT: `POST https://ec.europa.eu/taxation_customs/vies/rest-api/vat-validation` (CSV upload), `GET https://ec.europa.eu/taxation_customs/vies/rest-api/vat-

`validation/{token}` (status), `GET https://ec.europa.eu/taxation_customs/vies/rest-api/vat-validation-report/{token}` (Excel report).

Running the batch

1. Install dependencies (preferably in a venv):

   ```
   python -m venv .venv
   .\.venv\Scripts\activate
   pip install -r requirements.txt
   ```

2. Configure `.env` with the correct paths (sample values are present in the repo).
3. Run:

   ```
   python main.py
   ```

   This creates a timestamped folder under `DIRECTORY_LOCATION` with `siren_siret` and `vat` subfolders. Network access is required for INSEE and VIES calls.

Email notifications

- `emailing/mail_export.py` is invoked automatically at the end of `main.py` and sends reports from `masterdata@snetor.com` using the Microsoft Graph API. Ensure `emailing/config.cfg` points to the correct tenant/client IDs and PFX certificate (`MDMPythonGraphV2.pfx` by default).